Analyse de Mimikatz

Bonjour et bienvenue dans cette nouvelle analyse de binaire !

Nous allons aujourd'hui analyser un binaire bien connu du monde de la cybersécurité : j'ai nommé Mimikatz !

Avant toute chose, qu'est ce que Mimikatz ? Mimikatz est outil permettant de dump les identifiants et les hash/tokens d'identification contenus dans le processus Windows « lsass.exe ». Lsass est le responsable local des identifiants sur Windows.

Cela permets des élévation de privilèges et du déplacement latéral en environnement Active Directory, ce qui est très puissant et permissif.

Comme pour le cas de Meterpreter que nous avons vu précédemment, nous allons ici faire la lumière sur ce binaire qui ne cesse de faire ses preuves encore aujourd'hui !

Au menu : Analyse dynamique dans des sandbox et threat intelligence, ensuite analyse statique, et pour finir reverse-engineering.

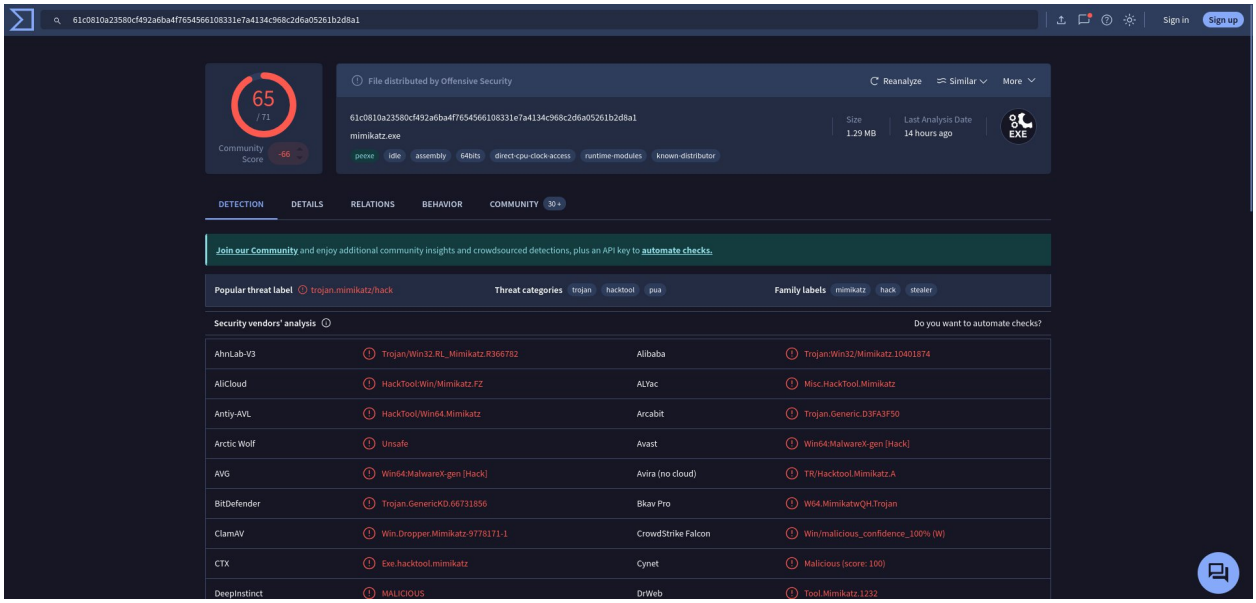
C'est parti !

Partie 1 : Les présentations

En premier lieu, et comme toujours dans mes analyses, nous allons récupérer le hash sha256 du binaire :

```
┌──(kali㉿kali)-[~/malwares]
└─$ sha256sum mimikatz.exe
61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1  mimikatz.exe
```

Maintenant que c'est fait, analysons le avec l'aide de virustotal :



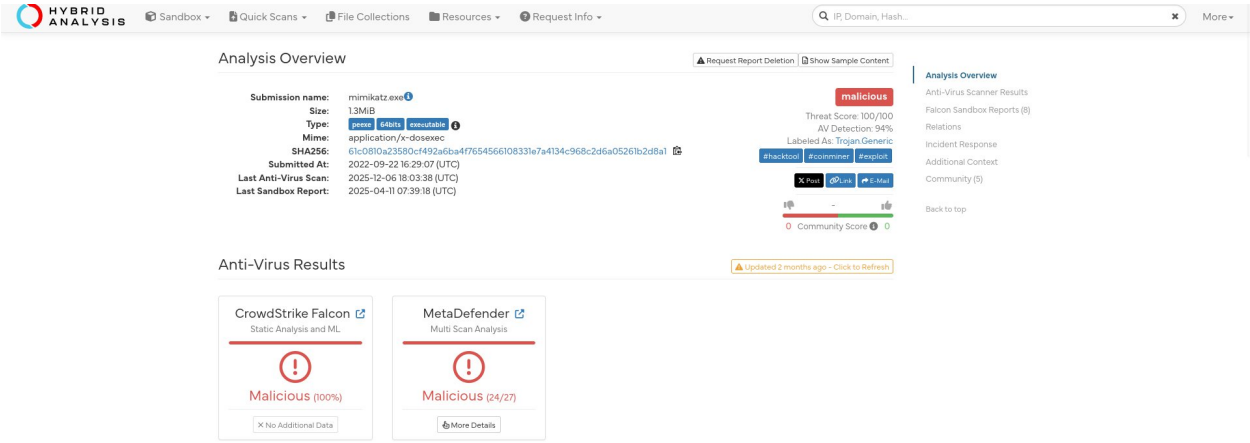
Comme je m'en doutais, le binaire est très connu, et fort heureusement, car ce qu'il permets de faire est vraiment dangereux.

En analysant les détails et relations du binaire sur virustotal, nous voyons qu'il est très souvent renommé pour le rendre plus furtif et tromper l'utilisateur, et nous voyons également qu'il est souvent lancé via un autre binaire comme powershell, ce qui est très courant dans le cas d'attaques « living on the land » envers les machines Windows.

Pour voir l'analyse virustotal en détails, je vous partage le lien de cette dernière :

https://www.virustotal.com/gui/file/61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1/detection

Ensuite, nous analysons le malware avec hybrid analysis, et heureusement comme pour virustotal, il est correctement détecté :



Lien de l'analyse : https://hybrid-analysis.com/sample/61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1

Nous ne nous attarderons pas plus sur ces analyses, car les rapports peuvent être lus à tout moment, nous allons maintenant rentrer dans le vif du sujet !

Partie 2 : Analyse statique

Pour commencer cette analyse statique, nous allons prendre plus d'informations sur le fichier :

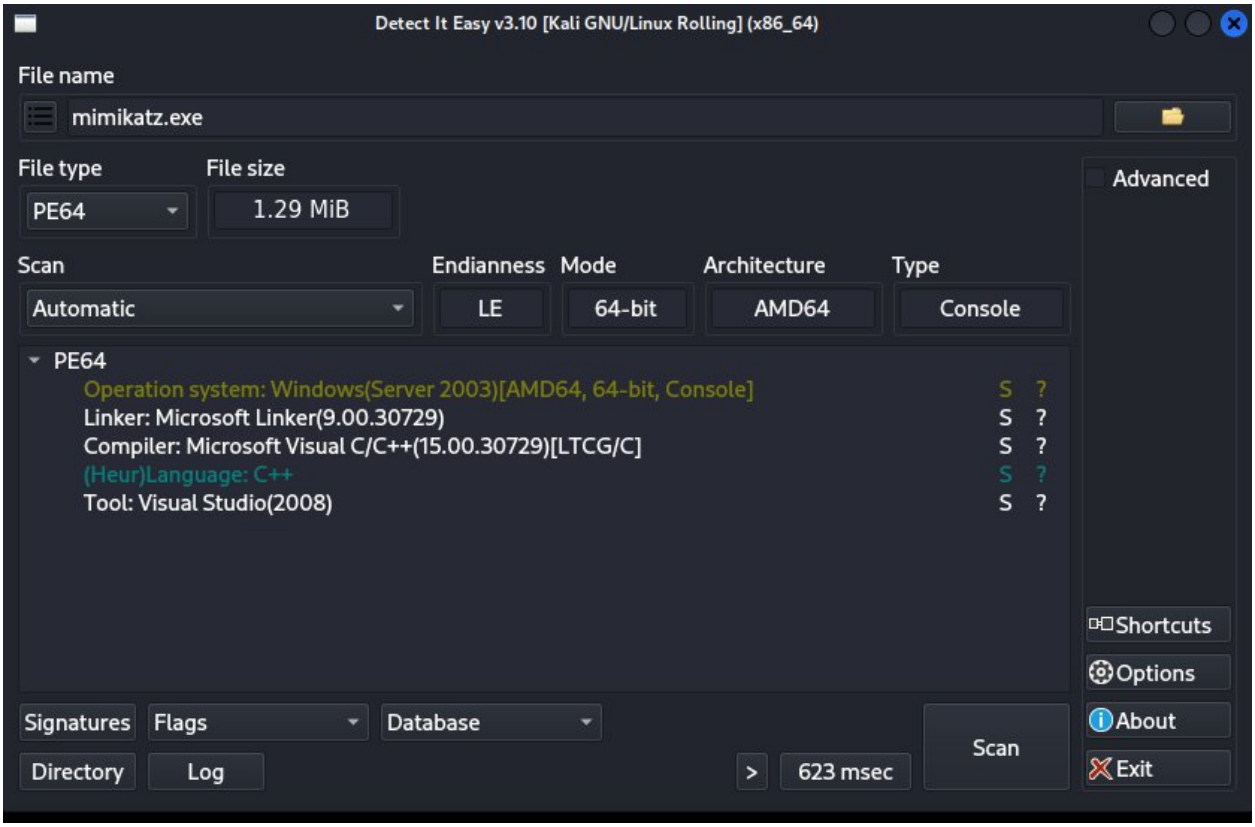
```
┌──(kali㉿kali)-[~/malwares]
└─$ file mimikatz.exe
mimikatz.exe: PE32+ executable for MS Windows 5.02 (console), x86-64, 6 sections
```

Ici nous avons la confirmation qu'il s'agit d'un PE Windows, sans surprise :)

Une analyse heuristique du binaire avec « diec » nous donne plus d'informations sur ce dernier, nous apprenons par exemple qu'il est écrit en C++ !

```
┌──(kali㉿kali)-[~/malwares]
└─$ diec --heuristicscan --verbose mimikatz.exe
[HEUR/About] Generic Heuristic Analysis by DosX (@DosX_dev)
[HEUR] Scanning has begun!
[HEUR] Scanning to programming language has started!
[HEUR/Any] Mangler detected → "msvcrt.dll", at function "terminate"
[HEUR/Any] C library present → "msvcrt.dll"
[HEUR/Any] C++ language detected!
[HEUR] Scan completed.
PE64
    Operation system: Windows(Server 2003)[AMD64, 64-bit, Console]
    Linker: Microsoft Linker(9.00.30729)
    Compiler: Microsoft Visual C/C++(15.00.30729)[LTCG/C]
    (Heur)Language: C++
    Tool: Visual Studio(2008)
```

Ensuite, nous passons notre ami Mimikatz dans l'outil « detect it easy / die » afin de voir si il est packé :



Le fichier n'est pas packé !

Ensuite, nous prenons des informations supplémentaires sur la structure du fichier via objdump :

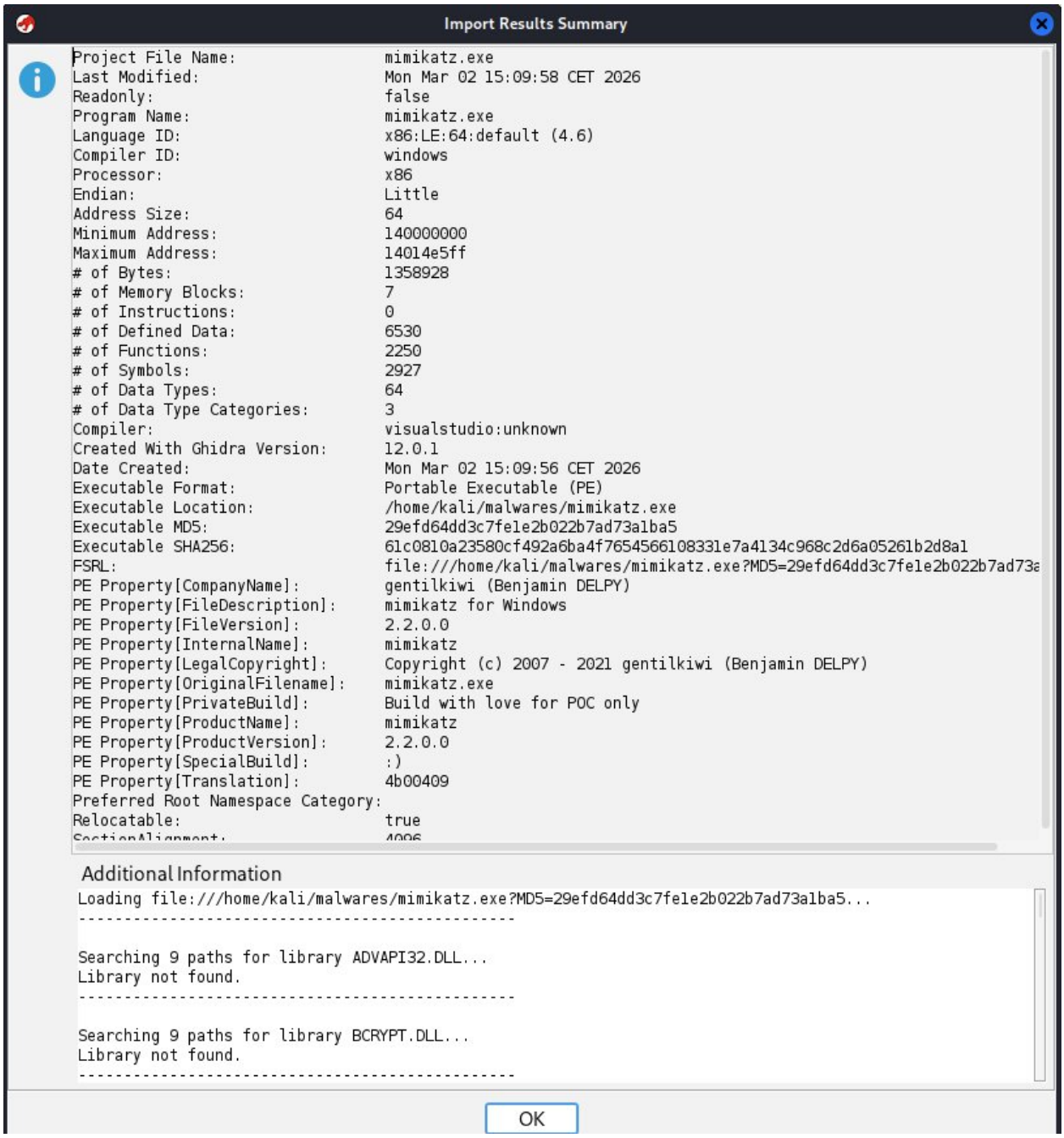
```
┌──(kali㉿kali)-[~/malwares]
└─$ objdump -f mimikatz.exe

mimikatz.exe:     format de fichier pei-x86-64
architecture: i386:x86-64, fanions 0x0000012f:
HAS_RELOC, EXEC_P, HAS_LINENO, HAS_DEBUG, HAS_LOCALS, D_PAGED
adresse de départ 0x00000001400c98e8


┌──(kali㉿kali)-[~/malwares]
└─$ objdump -h mimikatz.exe

mimikatz.exe:     format de fichier pei-x86-64

Sections :
Idx Name          Taille    VMA               LMA               Off fich  Algn
  0 .text         000cf709  0000000140001000  0000000140001000  00000400  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, CODE
  1 .rdata        00067768  00000001400d1000  00000001400d1000  000cfc00  2**4
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .data         00006a00  0000000140139000  0000000140139000  00137400  2**4
                  CONTENTS, ALLOC, LOAD, DATA
  3 .pdata        00006810  0000000140141000  0000000140141000  0013de00  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .rsrc         00003fe8  0000000140148000  0000000140148000  00144800  2**2
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .reloc        00002540  000000014014c000  000000014014c000  00148800  2**2
                  CONTENTS, LOAD, READONLY, DATA

┌──(kali㉿kali)-[~/malwares]
└─$ objdump -p mimikatz.exe | grep "DLL Name"

┌──(kali㉿kali)-[~/malwares]
└─$ objdump -p mimikatz.exe | grep "Import"
Entry 1 0000000000134530 0000026c Import Directory [parts of .idata]
Entry b 0000000000000000 00000000 Bound Import Directory
Entry c 00000000000d1000 000013c0 Import Address Table Directory
Entry d 0000000000134058 00000060 Delay Import Directory
        000d1050  <none>  009f  CryptImportKey
```

Synthèse de l’analyse statique :

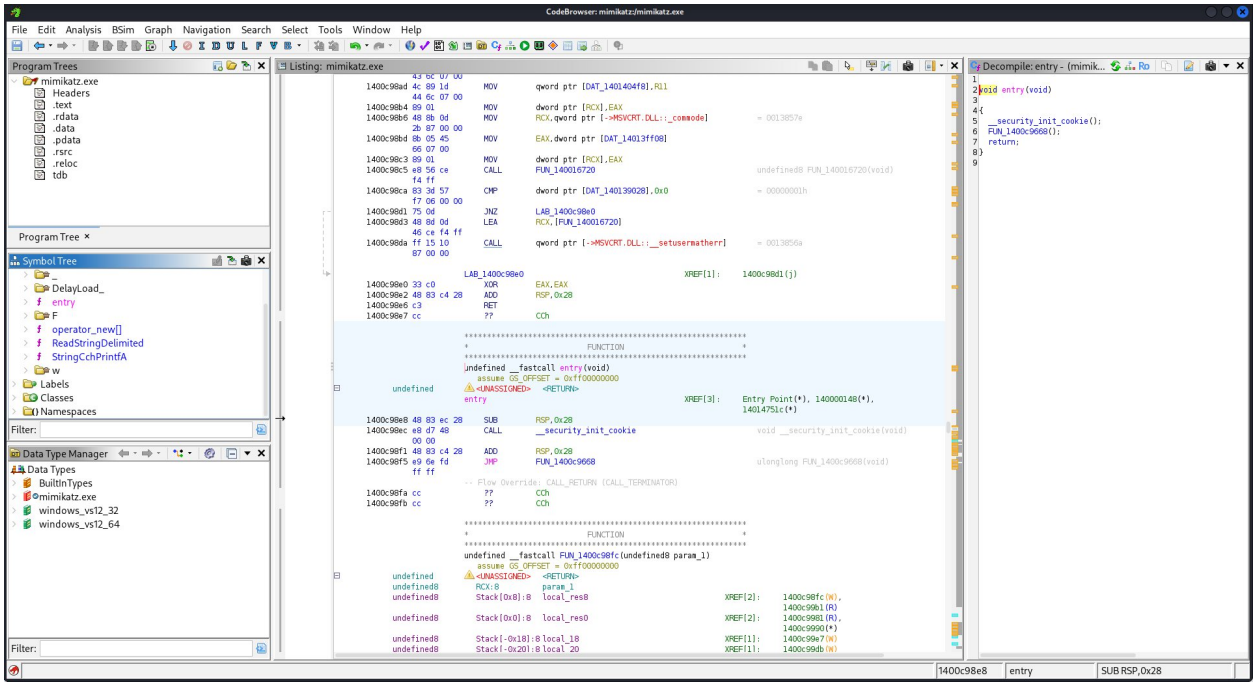
L’analyse statique préliminaire du binaire mimikatz.exe, révèle qu’il s’agit d’un exécutable Windows au format PE 64 bits (PEI-x86-64) destiné aux architectures x86-64. Les flags indiquent la présence de relocations (compatibilité ASLR), ainsi que des informations de debug et de symboles locaux, suggérant un binaire non totalement « strip », ce qui facilite le reverse engineering. La structure des sections (.text, .rdata, .data, .pdata, .rsrc, .reloc) est cohérente avec un exécutable légitime et ne montre aucun signe évident de packing ou d’obfuscation (absence de sections anormales ou de noms suspects). La section .text est particulièrement volumineuse (~850 Ko), ce qui reflète la richesse fonctionnelle de l’outil. La présence d’un Import Directory, d’une Import Address Table et d’un mécanisme de Delay Import confirme l’utilisation classique d’API Windows, et la détection de la fonction CryptImportKey suggère l’usage de mécanismes cryptographiques natifs, cohérents avec des opérations de manipulation et de déchiffrement de secrets système (notamment via DPAPI). À ce stade, rien n’indique un mécanisme de protection ou d’obfuscation avancé ; le binaire apparaît comme un outil natif complexe, modulaire et non packé, prêt à être analysé plus en profondeur via reverse engineering sous Ghidra afin d’identifier précisément les routines liées à l’extraction de credentials et à l’interaction avec les processus sensibles du système.

Partie 3 : Reverse-Engineering

Une fois le fichier importé dans Ghidra, le logiciel nous renvoie comme d'habitude des informations intéressantes :

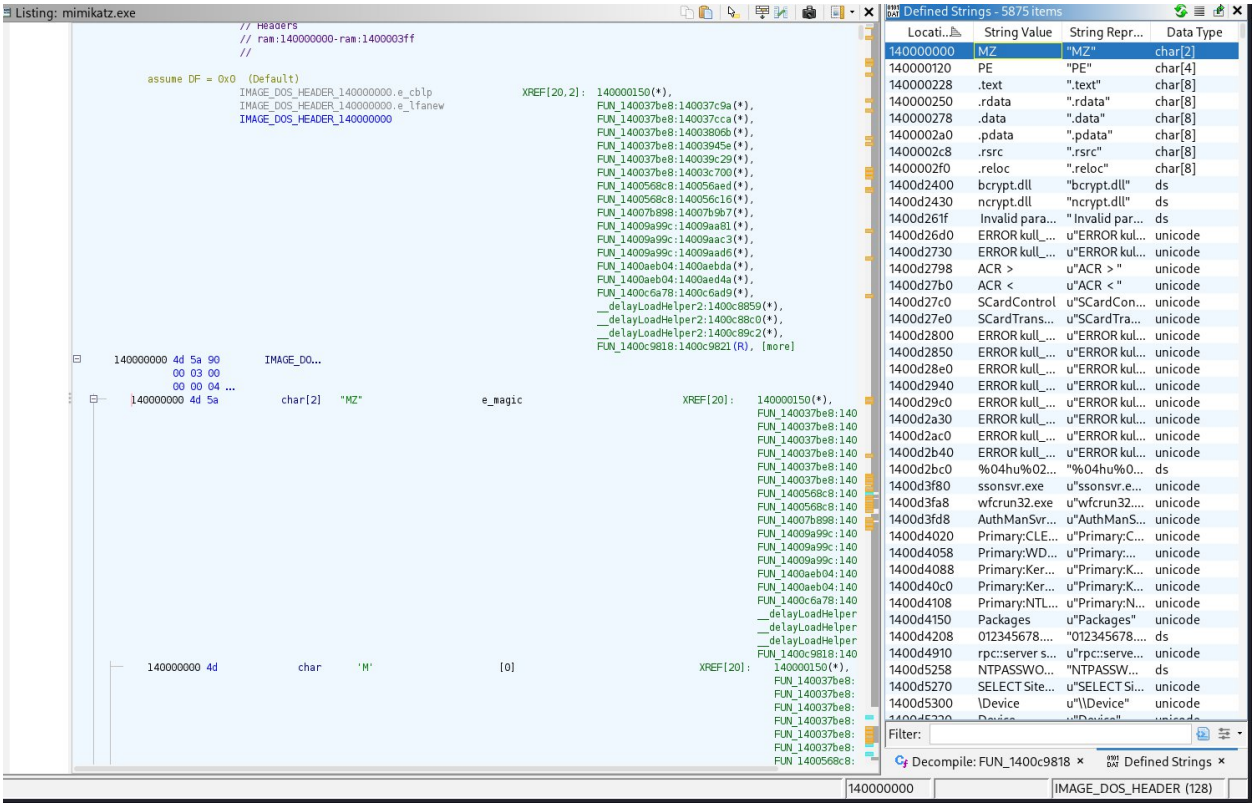


Voici la vue que nous avons suite à l'analyse du binaire dans Ghidra !



La capture montre l'analyse du binaire **mimikatz.exe** dans Ghidra au niveau de la fonction entry. On observe un point d'entrée très minimaliste et typique d'un exécutable compilé avec Microsoft Visual C++ : l'exécution commence par l'appel à __security_init_cookie(), mécanisme standard de protection contre les débordements de pile, puis est immédiatement redirigée vers une fonction interne identifiée comme FUN_1400c9668, qui correspond probablement à la véritable fonction principale (équivalent de main/wmain), mais nous devons le vérifier. L'absence de stub de décompression, de boucle de déchiffrement ou d'allocation mémoire suspecte à ce stade confirme que le binaire n'est pas packé et ne met pas en œuvre de mécanisme d'obfuscation ou d'anti-analyse au niveau de l'entrypoint. La structure observée est donc cohérente avec un exécutable natif MSVC classique, dont la logique métier principale se trouve dans la fonction appelée immédiatement après l'initialisation de sécurité.

Je fait ensuite une analyse des strings de la fonction « FUN_1400c9668 » qui s'apparente à la fonction main dans ce programme.



Cette fonction agit comme bootstrap du runtime C/C++. Elle commence par sauvegarder les registres et préparer la pile, puis effectue un verrouillage atomique sur une variable globale (DAT_140140508) pour éviter une double initialisation. Si le verrou est déjà pris, elle boucle en appelant Sleep pour attendre que l'init soit possible. Ensuite, elle vérifie l'état de l'initialisation (via DAT_140140500) et appelle _initterm pour exécuter les constructeurs globaux ou statiques du programme. Après cette initialisation, elle appelle finalement FUN_1400854b4, qui correspond probablement au main ou point d'entrée principal de l'application. La fonction se termine en restaurant les registres et en retournant proprement.

Analyse de la fonction « FUN_1400854b4 » :

```
Listing: mimikatz.exe - (3 addresses selected)
1400854ac 48 8d 05        LEA        RAX,[DAT_1400f7cd0]                    = 32h    2
          1d 28 07 00
1400854b3 c3              RET

                          **************************************************************
                          *                          FUNCTION                          *
                          **************************************************************
                          undefined __fastcall FUN_1400854b4(int param_1, longlong...
                            assume GS_OFFSET = 0xff00000000
          undefined       <UNASSIGNED>   <RETURN>
          int               ECX:4          param_1
          longlong          RDX:8          param_2
          undefined *       R8:8           param_3
          undefined8        R9:8           param_4
          undefined8        Stack[0x20]:8  local_res20                XREF[1]:     1400854c3(W)
          undefined8        Stack[0x18]:8  local_res18                XREF[1]:     1400854bf(W)
          undefined8        Stack[0x10]:8  local_res10                XREF[1]:     1400854bb(W)
          undefined8        Stack[0x8]:8   local_res8                 XREF[1]:     1400854b7(W)
                          FUN_1400854b4                        XREF[2]:     FUN_1400c9668:1400c97a0(c),
                                                                            14014571c(*)
1400854b4 48 8b c4        MOV        RAX,RSP
1400854b7 48 89 58 08     MOV        qword ptr [RAX + local_res8],RBX
1400854bb 48 89 68 10     MOV        qword ptr [RAX + local_res10],RBP
1400854bf 48 89 70 18     MOV        qword ptr [RAX + local_res18],RSI
1400854c3 48 89 78 20     MOV        qword ptr [RAX + local_res20],RDI
1400854c7 41 56           PUSH       R14
1400854c9 b8 20 00        MOV        EAX,0x20020
          02 00
1400854ce e8 3f ae        CALL       NTDLL.DLL::__chkstk                    undefined __chkstk()
          04 00
1400854d3 48 2b e0        SUB        RSP,RAX
1400854d6 48 63 f9        MOVSXD     RDI,param_1
1400854d9 48 8b 0d        MOV        param_1,qword ptr [->MSVCRT.DLL::_iob]  = 00138468
          28 cc 04 00
1400854e0 45 33 f6        XOR        R14D,R14D
1400854e3 48 83 c1 30     ADD        param_1,0x30
1400854e7 48 8b ea        MOV        RBP,param_2
1400854ea 41 8b de        MOV        EBX,R14D
1400854ed ff 15 8d        CALL       qword ptr [->MSVCRT.DLL::_fileno]       = 0013845e
          cc 04 00
1400854f3 be 00 00        MOV        ESI,0x40000
          04 00
1400854f8 8b c8           MOV        param_1,EAX
1400854fa 8b d6           MOV        param_2,ESI
1400854fc ff 15 b6        CALL       qword ptr [->MSVCRT.DLL::_setmode]      = 0013847c
          cb 04 00
140085502 48 8b 0d        MOV        param_1,qword ptr [->MSVCRT.DLL::_iob]  = 00138468
          ff cb 04 00
140085509 48 83 c1 60     ADD        param_1,0x60
14008550d 89 05 25        MOV        dword ptr [DAT_140140838],EAX
          b3 0b 00
140085513 ff 15 67        CALL       qword ptr [->MSVCRT.DLL::_fileno]       = 0013845e
          cc 04 00
140085519 8b d6           MOV        param_2,ESI
14008551b 8b c8           MOV        param_1,EAX
```

La fonction FUN_1400854b4 correspond au point d'entrée principal (main) de l'application. Elle commence par configurer l'environnement console : modification du mode des flux standards via _setmode, récupération et modification du code page avec GetConsoleOutputCP et SetConsoleOutputCP, définition du titre de la fenêtre via SetConsoleTitleW, puis installation d'un gestionnaire d'événements console (SetConsoleCtrlHandler). Elle affiche ensuite la bannière du programme ("mimikatz 2.2.0 x64") avant de déterminer le mode d'exécution.

<u>Deux modes sont implémentés :</u>

- Mode ligne de commande : si des arguments sont fournis (argc > 1), la fonction parcourt argv à partir de l'index 1, affiche chaque commande et appelle une fonction interne (FUN_1400857f4) chargée de traiter la commande.
- Mode interactif : en l'absence d'arguments, le programme entre dans une boucle REPL. Il affiche le prompt mimikatz #, lit l'entrée utilisateur via fgetws, nettoie le retour à la ligne, puis transmet la commande à la même fonction de dispatch.

La fonction gère également des codes de retour de type NTSTATUS (ex. 0xC000010A, 0xC000004B), ce qui indique que les sous-modules internes travaillent avec des statuts natifs Windows. En résumé, cette fonction ne contient pas la logique d'extraction de secrets mais implémente l'orchestrateur central, responsable de l'initialisation, de l'interface utilisateur et du dispatch des commandes vers les modules internes.

Analyse de la fonction « FUN_1400857f4 » :

```
Listing: mimikatz.exe

                    **************************************************************
                    *                          FUNCTION                          *
                    **************************************************************
                    uint __fastcall FUN_1400857f4(LPCWSTR param_1, undefined...
                      assume GS_OFFSET = 0xff00000000
        uint            EAX:4          <RETURN>
        LPCWSTR         RCX:8          param_1
        undefined8      RDX:8          param_2
        undefined8      R8:8           param_3
        undefined8      R9:8           param_4
        undefined8      Stack[0x18]:8  local_res18                XREF[2]:     1400857fe(W),
                                                                               1400858a7(R)
        undefined8      Stack[0x10]:8  local_res10                XREF[2]:     1400857f9(W),
                                                                               1400858a2(R)
        undefined8      Stack[0x8]:8   local_res8                 XREF[2]:     1400857f4(W),
                                                                               14008589d(R)
                    FUN_1400857f4                          XREF[5]:     FUN_1400854b4:1400855aa(c),
                                                                        FUN_1400854b4:140085642(c),
                                                                        FUN_1400b0508:1400b0606(c),
                                                                        FUN_1400b0740:1400b07ae(c),
                                                                        14014574c(*)
1400857f4 48 89 5c        MOV        qword ptr [RSP + local_res8],RBX
          24 08
1400857f9 48 89 6c        MOV        qword ptr [RSP + local_res10],RBP
          24 10
1400857fe 48 89 74        MOV        qword ptr [RSP + local_res18],RSI
          24 18
140085803 57              PUSH       RDI
140085804 48 83 ec 20     SUB        RSP,0x20
140085808 48 8b f1        MOV        RSI,param_1
14008580b bf 01 00        MOV        EDI,0xc0000001
          00 c0
140085810 48 85 c9        TEST       param_1,param_1
140085813 0f 84 84        JZ         LAB_14008589d
          00 00 00
140085819 45 33 c0        XOR        param_3,param_3
14008581c 33 d2           XOR        param_2,param_2
14008581e ff 15 5c        CALL       qword ptr [->KERNEL32.DLL::ExpandEnvironmentSt... = 00137ba6
          bd 04 00
140085824 8b e8           MOV        EBP,EAX
140085826 85 c0           TEST       EAX,EAX
140085828 74 73           JZ         LAB_14008589d
14008582a 48 8b d5        MOV        param_2,RBP
14008582d b9 40 00        MOV        param_1,0x40
          00 00
140085832 48 03 d2        ADD        param_2,param_2
140085835 ff 15 0d        CALL       qword ptr [->KERNEL32.DLL::LocalAlloc]          = 001379e4
          be 04 00
14008583b 48 8b d8        MOV        RBX,RAX
14008583e 48 85 c0        TEST       RAX,RAX
140085841 74 5a           JZ         LAB_14008589d
140085843 44 8b c5        MOV        param_3,EBP
140085846 48 8b d0        MOV        param_2,RAX
140085849 48 8b ce        MOV        param_1,RSI
14008584c ff 15 2e        CALL       qword ptr [->KERNEL32.DLL::ExpandEnvironmentSt... = 00137ba6
```

La fonction FUN_1400857f4 agit comme dispatcher central des commandes. Elle reçoit en paramètre une chaîne Unicode représentant une commande utilisateur (issue soit des arguments CLI, soit du mode interactif). La fonction commence par valider le pointeur puis effectue une expansion des variables d'environnement via ExpandEnvironmentStringsW. Pour cela, elle appelle d'abord l'API afin de déterminer la taille nécessaire, alloue dynamiquement un buffer avec LocalAlloc, puis réexécute l'expansion dans ce buffer. Cela permet à l'outil de supporter des commandes contenant des variables telles que %TEMP% ou %USERPROFILE%.

Une fois la chaîne correctement expandée, la fonction examine le premier caractère afin de déterminer le type de traitement à appliquer. Si la commande commence par !, elle est redirigée vers un handler spécifique (FUN_14009e2b0). Si elle commence par *, elle est envoyée vers un

autre handler dédié (FUN_1400af6e4). Dans le cas général (aucun préfixe particulier), la commande est transmise au parser principal (FUN_1400858b4), qui correspond très probablement au mécanisme de résolution des commandes de type module::action. Après exécution du handler approprié, le buffer alloué est libéré via LocalFree, et la fonction retourne le code NTSTATUS produit par le module appelé. Par défaut, le code retourné est 0xC0000001 (échec générique), confirmant que l'ensemble du moteur interne repose sur des codes de statut Windows natifs.

Analyse de la fonction « FUN_1400858b4 » :

```
Listing: mimikatz.exe
1400858b3 c3          RET

                      **************************************************************
                      *                          FUNCTION                          *
                      **************************************************************
                      wchar_t * __fastcall FUN_1400858b4(LPCWSTR param_1, unde...
                         assume GS_OFFSET = 0xff00000000
        wchar_t *        RAX:8          <RETURN>
        LPCWSTR          RCX:8          param_1
        undefined8       RDX:8          param_2
        size_t           R8:8           param_3
        undefined8       R9:8           param_4
        undefined4       Stack[0x20]:4  local_res20                XREF[3]:     1400858d2(W),
                                                                                 140085a48(W),
                                                                                 140085bee(R)
        undefined4       Stack[0x18]:4  local_res18                XREF[3]:     1400858cc(*),
                                                                                 1400858fd(R),
                                                                                 140085a2a(R)
        undefined4       Stack[0x10]:4  local_res10                XREF[5]:     1400858f0(W),
                                                                                 1400859e2(R),
                                                                                 140085a1e(W),
                                                                                 140085a44(R),
                                                                                 140085aee(R)
        undefined8       Stack[0x8]:8   local_res8                 XREF[2]:     1400858b7(W),
                                                                                 140085bf7(R)
        undefined8       Stack[-0x40]:8 local_40                   XREF[2]:     140085a07(W),
                                                                                 140085a31(R)
        undefined8       Stack[-0x48]:8 local_48                   XREF[5]:     1400858e2(W),
                                                                                 140085941(W),
                                                                                 140085a5b(R),
                                                                                 140085ae4(R),
                                                                                 140085bc4(R)
                      FUN_1400858b4                          XREF[2]:     FUN_1400857f4:140085877(c),
                                                                          140145758(*)
1400858b4 48 8b c4       MOV      RAX,RSP
1400858b7 48 89 58 08    MOV      qword ptr [RAX + local_res8],RBX
1400858bb 55             PUSH     RBP
1400858bc 56             PUSH     RSI
1400858bd 57             PUSH     RDI
1400858be 41 54          PUSH     R12
1400858c0 41 55          PUSH     R13
1400858c2 41 56          PUSH     R14
1400858c4 41 57          PUSH     R15
1400858c6 48 83 ec 30    SUB      RSP,0x30
1400858ca 33 ed          XOR      EBP,EBP
1400858cc 48 8d 50 18    LEA      param_2=>local_res18,[RAX + 0x18]
1400858d0 8b fd          MOV      EDI,EBP
1400858d2 89 ac 24       MOV      dword ptr [RSP + local_res20],EBP
          88 00 00 00
1400858d9 ff 15 19       CALL     qword ptr [->SHELL32.DLL::CommandLineToArgvW]    = 00136fd4
          c3 04 00
1400858df 48 8b f5       MOV      RSI,RBP
1400858e2 48 89 6c       MOV      qword ptr [RSP + local_48],RBP
          24 20
1400858e7 4c 8b f5       MOV      R14,RBP
```

La fonction FUN_1400858b4 constitue le cœur du moteur de résolution des commandes de Mimikatz. Elle agit comme un parser et un dispatcher central chargé d'interpréter les commandes saisies par l'utilisateur sous la forme module::commande. Dans un premier temps, elle transforme la chaîne reçue en un tableau d'arguments via CommandLineToArgvW, ce qui lui permet de manipuler les entrées comme un véritable programme en ligne de commande. Elle recherche ensuite la présence du séparateur : afin d'isoler le nom du module de la commande proprement dite. Si ce séparateur est trouvé, la partie gauche est copiée dans un buffer alloué dynamiquement et utilisée pour identifier le module ; sinon, la commande est traitée selon le comportement par défaut.

La fonction parcourt alors une table globale statique contenant l'ensemble des modules disponibles. Chaque module est décrit par une structure regroupant son nom, son nom complet, sa description, le nombre de commandes qu'il expose et un tableau de pointeurs vers ces commandes. La recherche s'effectue via des comparaisons Unicode insensibles à la casse (_wcsicmp). Une fois le module correspondant identifié, une seconde boucle parcourt son tableau de commandes afin de trouver celle qui correspond à l'entrée utilisateur. Si une correspondance est détectée, la fonction déclenche un appel indirect vers le handler associé à l'aide d'un pointeur de fonction, ce qui confirme une architecture modulaire reposant sur des tables statiques et un dispatch dynamique.

En cas d'échec, la fonction gère proprement les erreurs : si le module est inconnu, elle affiche la liste des modules disponibles ; si le module existe mais que la commande est invalide, elle affiche les informations détaillées du module ainsi que les commandes qu'il expose. Enfin, elle libère les buffers alloués et retourne le code de statut produit par le handler appelé. Ainsi, FUN_1400858b4 ne contient aucune logique d'extraction de secrets à proprement parler, mais constitue le pivot architectural du programme, assurant la liaison entre l'interface utilisateur et les différents modules internes.

Conclusion sur le reverse-enginering de Mimikatz :

L'analyse statique du binaire mimikatz.exe dans Ghidra montre d'abord un point d'entrée minimaliste et typique d'un exécutable compilé avec Microsoft Visual C++. L'exécution débute par un appel à __security_init_cookie(), mécanisme standard de protection contre les débordements de pile, puis est immédiatement redirigée vers une fonction interne identifiée comme FUN_1400c9668. L'absence de stub de décompression, de routine de déchiffrement ou de mécanisme d'anti-analyse à l'entrypoint confirme que le binaire n'est ni packé ni obfusqué à ce niveau. La structure observée correspond donc à un exécutable natif MSVC classique.

La fonction FUN_1400c9668 agit comme bootstrap du runtime C/C++. Elle met en place l'environnement d'exécution en protégeant l'initialisation via un verrou global afin d'éviter toute double initialisation. Elle appelle ensuite _initterm pour exécuter les constructeurs globaux et statiques, avant de transférer le contrôle vers FUN_1400854b4, qui correspond au véritable point d'entrée logique du programme (équivalent d'un main).

La fonction FUN_1400854b4 met en place l'environnement console : configuration des flux standards avec _setmode, gestion du code page via GetConsoleOutputCP et SetConsoleOutputCP, définition du titre de la fenêtre et installation d'un handler de contrôle console. Elle affiche ensuite la bannière du programme ("mimikatz 2.2.0 x64") puis détermine le mode d'exécution. Deux modes sont implémentés : un mode ligne de commande si des arguments sont fournis, et un mode interactif de type REPL si aucun argument n'est présent. Dans les deux cas, les commandes sont transmises à une fonction de dispatch dédiée (FUN_1400857f4). Cette fonction joue un rôle d'orchestrateur : elle ne contient aucune logique d'extraction de secrets, mais centralise l'initialisation et la gestion des entrées utilisateur.

FUN_1400857f4 agit comme premier niveau de traitement des commandes. Elle valide l'entrée, effectue une expansion des variables d'environnement via ExpandEnvironmentStringsW, puis redirige la commande vers différents handlers en fonction de son préfixe (!, *, ou aucun). Dans le

cas général, la commande est transmise à FUN_1400858b4, qui constitue le cœur du moteur de résolution interne.

La fonction FUN_1400858b4 implémente le parser et le dispatcher principal des commandes de type module::commande. Elle commence par convertir la chaîne reçue en tableau argc/argv via CommandLineToArgvW. Elle recherche ensuite le séparateur : afin de distinguer le nom du module de la commande. Si le séparateur est présent, la partie gauche est copiée dynamiquement dans un buffer pour identifier le module ; sinon, un comportement par défaut est appliqué.

La fonction parcourt alors une table globale statique contenant l'ensemble des modules disponibles. Chaque module est représenté par une structure comprenant son nom, son nom complet, sa description, le nombre de commandes qu'il expose et un tableau de commandes. Les comparaisons s'effectuent via _wcsicmp, donc sans sensibilité à la casse. Une fois le module identifié, une seconde boucle parcourt la liste des commandes du module pour trouver celle correspondant à l'entrée utilisateur. Si une correspondance est trouvée, le handler associé est invoqué via un appel indirect par pointeur de fonction.

Ce mécanisme met en évidence une architecture modulaire basée sur des tables statiques et un dispatch dynamique. Il n'y a pas de résolution par hash, pas d'obfuscation du flux de contrôle ni de mécanisme de dissimulation avancé à ce niveau. La logique repose sur des structures clairement définies en mémoire et des appels indirects standards. En cas d'erreur (module inconnu ou commande invalide), la fonction affiche des messages explicites ainsi que la liste des modules ou des commandes disponibles, avant de libérer les buffers alloués.

Dans son ensemble, l'analyse met en lumière une architecture interne propre, structurée et maintenable. Le binaire repose sur une séparation nette des responsabilités : initialisation runtime, configuration de l'environnement, gestion des entrées utilisateur, puis dispatch modulaire vers des handlers spécialisés. Les fonctions analysées ne contiennent pas directement la logique d'extraction de secrets, mais constituent le socle organisationnel permettant d'atteindre ces fonctionnalités.

Ainsi, le reverse engineering effectué révèle que le cœur de l'outil ne repose pas sur la dissimulation, mais sur une architecture modulaire déclarative, centralisée autour d'un moteur de résolution de commandes clair et facilement cartographiable.

Conclusion globale sur cette analyse :

En conclusion, l'analyse statique et le reverse engineering montrent que Mimikatz repose sur une architecture modulaire claire et non obfusquée. Le binaire n'est pas packé, suit un schéma classique MSVC et sépare strictement l'initialisation, l'interface console et le moteur de dispatch des commandes. Le cœur du programme est un routeur interne qui résout les commandes de type module::action via une table statique de modules et des appels indirects par pointeurs de fonctions. Autrement dit, mimikatz ne dissimule pas sa logique structurelle : il s'appuie sur une organisation propre et déclarative, où chaque module expose ses fonctionnalités, et où le moteur central agit simplement comme intermédiaire entre l'utilisateur et les handlers spécialisés.

Merci pour votre lecture !